Timing Predictability of the Critter Embedded System

Matthew Ketas
Department of Electrical and Computer Engineering
Swanson School of Engineering, University of Pittsburgh
Pittsburgh, PA, USA
mtk67@pitt.edu

Abstract—This study evaluates timing predictability in the Critter, a single-threaded Raspberry Pi 5 prototype for unattended machine-room temperature monitoring. Sixteen conditions vary online cores, CPU governor, system RAM, receive traffic, and scheduling preference under matrix-multiplication competition. The primary dataset contains three 60-second runs per condition with a 100 ms period and deadline. Five measures describe deadline compliance and severity, with median and p99 response reported separately. CPU competition produced the largest repeatable response-time increase: at Critter nice +19, mean run median response was 3.128 ms, versus 0.132 ms for the pinned quiet control, while every job still met its deadline. The largest observed deadline failure occurred in one 1.5 GiB run, with six missed slots, 245.159 ms maximum lateness, and a 444.746 ms acquisition gap. Its p99 was only 0.497 ms, demonstrating why a percentile alone cannot establish reliability. The evidence distinguishes dispatch interference from delays inside I/O, supports cautious evaluation of lower-resource Critter configurations, and connects bounded memory use and service priorities to broader embedded-system requirements. A pacemaker example explains why these measurements support architectural questions but cannot establish hard real-time suitability.

Keywords—embedded systems, Critter, timing predictability, soft real time, scheduling, memory pressure

I. Introduction

The largest impact on predictability depends on what the product must preserve. In these Critter experiments, ordinary CPU scheduling preference under competition most clearly changed the response-time distribution. A low-RAM condition produced the most severe observed interruption, including the only missed deadlines in the primary dataset. These findings answer different parts of the question: a slower but consistently timely job is different from a usually fast job that occasionally leaves a long gap in observation.

The course product monitors distributed machine rooms to help identify heating, ventilation, and air-conditioning (HVAC) problems. Its I/O unit collects temperatures, its memory unit removes outliers and summarizes history, and its computational unit predicts HVAC behavior. The assignment specifies soft rather than hard real-time requirements and asks which resource conditions most affect predictability [1]. I therefore evaluate whether the prototype delivers scheduled work on time and how much observation continuity is lost when it does not, rather than treating average execution speed as reliability.

The measurements and my interpretations are recorded in the annotated analysis workbook and underlying experiment archive [2]. The findings extend my previous power study, which identified lower-resource configurations without establishing timing compliance [3]. An implementation that uses less power is useful to Critter only if its temperature evidence remains sufficiently timely. This paper first defines the experiment, then distinguishes the measured mechanisms and their consequences for Critter. It closes by applying those lessons to embedded systems with more demanding timing obligations.

II. Experimental Method

A. Prototype and scheduled workload

The tested program reuses Group C's acquisition, sample-saving, memory-filtering, and prediction functions. During each periodic job, one thread acquires a temperature, saves it to a CSV file, and updates a 100-sample memory window using the existing outlier logic. Prediction and final summary construction occur after the collection interval, not concurrently inside each acquisition job. The implementation therefore represents the present sequential prototype; it does not emulate a future continuously intensive predictor.

All primary runs use the same fixed 22.5 C CSV input and the same executable and input hashes. The data reader opens the file and returns its first numeric temperature on each call. This is a repeatable timing fixture, not a sequential replay or a physical temperature-sensor test. Repeated file access, saving, and filtering remain in the measured path. Timing observations are preallocated and touched before collection, buffered during the run, and exported afterward. Sample saving still occurs during each job, and completion of a buffered write and close does not prove durable SD-card storage.

The study assumes a 100 ms period and a 100 ms relative deadline, yielding 600 scheduled slots in one minute. These are experimental requirements derived from the prototype's nominal 10 Hz rate; they were not supplied as physical HVAC limits. Releases are scheduled against CLOCK_MONOTONIC using absolute waits, so processing time does not continually shift the next release. Absolute sleeping avoids the accumulated schedule drift of repeated relative sleeps, but cannot guarantee immediate execution when the wakeup time arrives [4]. If processing runs past future releases, those slots are recorded as skipped rather than recovered through a burst of measurements.

The course sleeploop.c illustrates sleep and wakeup variability, but its usleep(500) argument requests 500 microseconds despite the variable name mSeconds. Its consecutive intervals also include surrounding work and output. The Critter wrapper instead keeps application demand fixed and

measures completion relative to the intended release. It does not oversample to compensate for late work.

B. Timing reliability and severity measures

Response time is completion time minus scheduled release time. It includes dispatch delay and elapsed work, including blocking or preemption within that work. A job meets its deadline only when acquisition, saving, and filtering succeed and completion is no later than release plus 100 ms. A legitimate outlier rejection is not an operation failure. Timing reliability here means compliance with this study contract; it is not an estimate of device lifetime or hardware failure probability.

The five measures are deadline-miss percentage; median and nearest-rank p99 response for attempted jobs; maximum observed lateness beyond the deadline; maximum gap between successful acquisitions, including interval boundaries; and the longest consecutive sequence of missed slots. Unsuccessful, late, and skipped jobs all count toward the miss rate and streak. A skipped release has no response-time observation, so it is not assigned an invented latency. The streak's slot count describes missed scheduled demand, while the acquisition gap measures the actual interval without a new observation.

GNU time supplies elapsed, user and system CPU time, peak resident memory, page faults, and context switches [5]. Those process-level measurements help interpret resource use, but the total elapsed time cannot identify which 100 ms deadlines were missed. The per-job trace is the deadline detector. A program that waits for most of a minute can have similar total elapsed time with and without a serious interruption inside that minute.

C. Conditions and controls

The common baseline has four online cores, native 8 GB RAM, performance governor, Critter nice 0, and no generated traffic or matrix competitor. Cores are reduced to three, two, and one using boot limits, with the online mask checked. These are system-wide online-core treatments, not merely restrictions on the application's affinity. An initial runtime CPU-hotplug attempt failed to restore CPU3, so the successful campaign uses separate boots and retains the failed attempt as a setup failure.

The governor sweep uses performance, powersave, and ondemand with the same 1.5–2.4 GHz policy bounds. Performance requests the upper permitted frequency, powersave the lower, and ondemand adapts to load [6]. The RAM sweep applies boot limits of 4, 2, and 1.5 GiB while retaining four cores, performance, and quiet networking. These limits constrain memory managed by the whole Linux system; they do not replace the DRAM hardware or isolate Critter's own allocation.

Networking uses one reverse UDP iperf3 stream from the Mac, with 1200-byte payloads and nominal receive rates of 10 and 50 Mbit/s [7]. Background traffic begins before measurement. The successful primary tests achieved 9.963–9.990 and 49.711–49.977 Mbit/s, respectively. The priority comparison first pins a quiet Critter to CPU0, then adds the instructor's repeating 1000-by-1000 integer matrix multiplication on that same CPU. The matrix worker remains nice 0; Critter is tested at nice -10, 0, +10, and +19. The matrix source is built without optimization so its unused numerical result is not removed.

Application and workload children use ordinary SCHED_OTHER scheduling and share the recorded autogroup and cgroup. Linux nice values influence scheduling preference within this environment; they do not reserve a processor or provide a hard completion guarantee [8]. The pinned quiet control distinguishes affinity from competition, and matrix nice 0 distinguishes competition from subsequently changing Critter's preference. Changing Critter's nice value affects its whole sequential pipeline, not an independently prioritized sampling thread.

D. Repetitions and evidence selection

The main figures use the completed 30-run native-RAM campaign plus six completed three-run core/RAM blocks: 48 runs, 16 conditions, and 28,800 scheduled slots. Conditions are shuffled within each native-RAM repetition; core and RAM treatments occur in successive boots. The archive also retains 18 valid runs from an earlier interrupted campaign, two failed main attempts, and 13 short smoke checks. The earlier complete runs remain separate to preserve a balanced three-run comparison. Inclusion depends on configuration and collection validity, not on obtaining zero misses. The low-RAM run with misses remains included.

All plotted metrics were recomputed from the raw traces and checked against the archived summaries. Means are averages of three whole-run statistics; mean p99 is not a pooled percentile. Observed ranges show repetition differences, not confidence intervals. The small sample supports descriptive comparisons rather than an inferential claim that every small difference is statistically resolved. Before and after every primary run, the Pi reported no recorded throttling flags.

III. Measured Results

Table I reports all sixteen conditions. Response values are means of three run-level statistics; misses are totals over 1,800 scheduled slots, and the gap is the worst of the three runs. Only the 1.5 GiB condition recorded deadline misses. This does not make all other settings equivalent: their response distributions differ even when the 100 ms deadline leaves substantial margin.

TABLE I
RESPONSE AND OBSERVATION CONTINUITY

Condition	Mean median (ms)	Mean p99 (ms)	Misses/ 1800	Worst gap (ms)
Baseline	0.135	0.157	0	100.072
3 cores	0.138	0.162	0	100.073
2 cores	0.149	0.178	0	102.703
1 core	0.146	0.221	0	100.359
Powersave	0.161	0.189	0	102.497
Ondemand	0.164	0.191	0	100.060

Condition	Mean median (ms)	Mean p99 (ms)	Misses/ 1800	Worst gap (ms)
RAM 4 GiB	0.143	0.197	0	103.128
RAM 2 GiB	0.131	0.239	0	103.540
RAM 1.5 GiB	0.128	0.339	6	444.746
Network 10M	0.138	0.159	0	100.061
Network 50M	0.140	0.164	0	100.069
Pinned quiet	0.132	0.159	0	100.073
Matrix nice -10	0.151	0.186	0	102.509
Matrix nice 0	0.151	2.582	0	102.530
Matrix nice +10	0.158	2.975	0	103.484
Matrix nice +19	3.128	3.162	0	103.990

Three runs per condition. Responses average run statistics; gap is the worst observed run. Native RAM and performance are supplied by baseline. Matrix stays nice 0. Source: [2].

A. Competition and scheduling preference

The pinned quiet control had mean run median response of 0.132 ms and mean run p99 of 0.159 ms. Adding the nice 0 matrix competitor raised the corresponding p99 to 2.582 ms while the median remained 0.151 ms. At Critter nice +10, mean p99 was 2.975 ms. At nice +19, both typical and tail behavior changed: mean median was 3.128 ms and mean p99 was 3.162 ms. The latter condition's run medians ranged from 2.648 to 4.061 ms. Fig. 1 shows the large separation from the quiet controls.

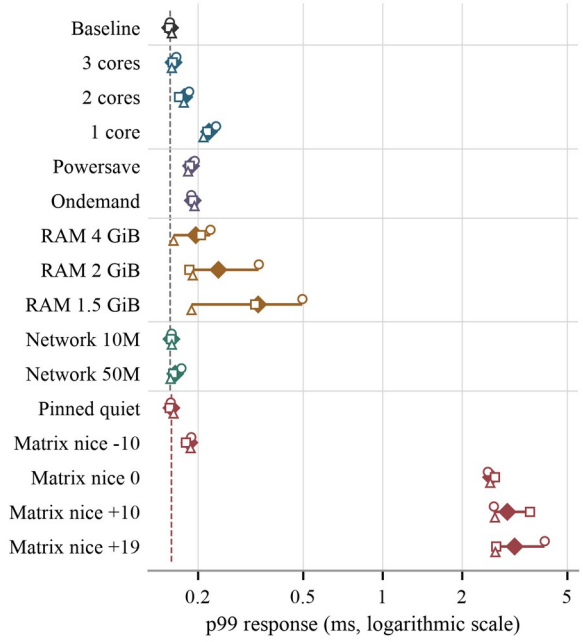


Fig. 1. Run p99 response across the sixteen conditions. Open circles, squares, and triangles show repetitions 1–3; diamonds show their mean. The logarithmic axis displays both sub-millisecond controls and millisecond competition delays. The dashed control changes to pinned quiet for the priority group.

Nice +19 increased mean median response by about 23.6 times relative to pinned quiet, yet all 1,800 jobs in that condition still met the deadline. Thus the result supports my concern about assigning sensing work the lowest tested scheduling preference, but not a claim that this preference caused deadline failure in the experiment. Conversely, giving Critter nice -10 under competition kept mean p99 at 0.186 ms. This was the highest preference tested, not the highest possible Linux preference or a real-time scheduling class.

Stage timing strengthens the interpretation. Under matrix competition at Critter nice 0, run p99 dispatch delays were 2.407 –2.576 ms, whereas p99 elapsed job-work times were 0.110–0.111 ms. At nice +19, dispatch-delay p99 ranged from 2.576 to 3.986 ms. These separate percentiles cannot be added to reconstruct response p99, but their magnitudes are consistent with CPU competition delaying when useful work begins. A fast algorithm does not ensure a prompt start.

B. The low RAM interruption

The first 1.5 GiB repetition missed six of its 600 scheduled slots, a 1.00% miss rate. Its other two repetitions had no misses; across that condition the total is six of 1,800 slots, or a mean run miss rate of 0.333%. Table II preserves that variation. The worst attempted response was 345.159 ms, maximum lateness was 245.159 ms, and the longest acquisition gap was 444.746 ms. Four consecutive missed slots represented 400 ms of scheduled demand. These quantities describe different aspects of the interruption and should not be treated as interchangeable outage durations.

TABLE II
SEVERITY IN THE THREE LOW RAM REPETITIONS

Run	Misses/ 600	p99 (ms)	Maximum lateness (ms)	Maximum gap (ms)	Longest streak (slots)
1	6	0.497	245.159	444.746	4
2	0	0.330	0.000	100.568	0
3	0	0.189	0.000	102.839	0

All other primary runs had zero maximum lateness and zero missed-slot streaks. Slot period is 100 ms. Source: [2].

The trace locates two late attempts. The job released at 13.1 s started only 0.072 ms late, then spent 106.061 ms in sample saving. The job released at 13.4 s started 0.058 ms late, then spent 345.015 ms in acquisition. Those intervals include any blocking or descheduling while inside the respective calls. The remaining four misses were skipped releases. Fig. 2 therefore illustrates a different mechanism from the competition experiment: substantial delay arose after work began, within the I/O path, rather than chiefly before dispatch.

This run's p99 response was only 0.497 ms. It contained 596 attempted jobs, of which only two exceeded 100 ms, and the four skipped slots do not enter the response distribution. A percentile covering almost all attempted jobs can therefore look excellent while the application loses an important interval of observation. The median was also low. My interpretation is that typical speed

and predictability of the rare interruption must be examined together.

Low available system memory is a plausible contributor, but capacity alone is not a demonstrated cause. The affected process reported two major page faults, and system-wide telemetry recorded swap and fault activity. Across all primary runs, GNU time reported peak Critter resident memory of 1,088–1,616 KiB, not approximately 1.3 GB; the affected run used 1,600 KiB. Resident size is not the whole system's memory demand. The boot cap also constrains the desktop, services, caches, and other processes, with zram swap already in use. Neither a precise remaining-memory threshold nor allocator causation follows from these records. The two unaffected repetitions make the event a reason for investigation rather than a repeatable failure rate established for every 1.5 GiB deployment.

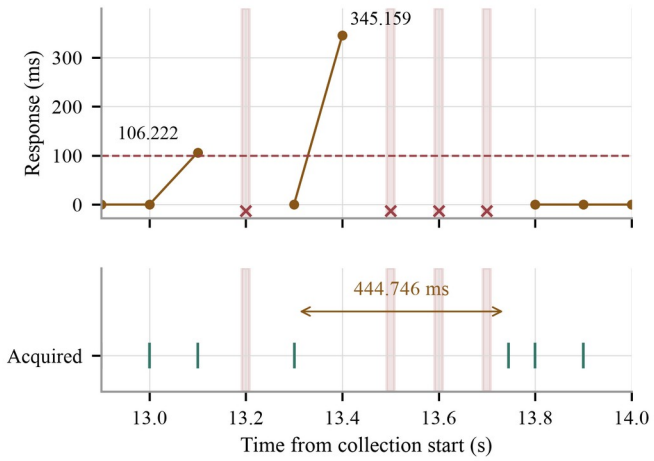


Fig. 2. Close-up of the first 1.5 GiB run. Two attempted jobs exceeded the dashed 100 ms deadline. Shaded releases were skipped and have no plotted response. Acquisition markers show a 444.746 ms gap. The job with the 345.159 ms response began close to its intended release.

C. Core count governors and networking

With one online core, mean median response was 0.146 ms and mean p99 was 0.221 ms, compared with 0.135 and 0.157 ms at baseline. Powersave produced 0.161 and 0.189 ms, and ondemand produced 0.164 and 0.191 ms. The relative changes are measurable in the descriptive data, but their absolute sizes are small compared with 100 ms. No misses occurred in any of these conditions. One thread cannot run its sequential functions on several cores simultaneously, although extra cores may still accommodate the operating system and other work.

The 10 and 50 Mbit/s receive conditions had mean p99 values of 0.159 and 0.164 ms, with no missed slots. Networking was therefore not the dominant timing influence at the tested rates. Across the other fifteen conditions, the largest observed acquisition gap was 103.990 ms, occurring at nice +19 under matrix competition. This was a small continuity change relative to the low-RAM event. These comparisons do not establish that one core combined with powersave and network activity will behave like each isolated treatment: that combination was not tested.

IV. Implications for the Critter Product

A. Define reliability through useful temperature evidence

For Critter, I would interpret a timing result through the temperature evidence an unattended operator needs. A 3 ms response may be entirely adequate for this study's 100 ms contract, even though it is much slower than baseline. A 445 ms acquisition gap may also be acceptable for a slowly changing room, but that decision requires the room's dynamics and the allowed warning delay. Soft real time permits a product-specific tolerance; it does not make every delay harmless or allow an average to hide an extended blind interval.

I would specify acquisition continuity, the maximum age of data used by a summary, and the delay before an HVAC concern is reported. These are related obligations rather than one completion statistic. Successful acquisition can still be followed by delayed saving or reporting. A timely program that repeatedly processes stale or incorrect input would not fulfill the product's purpose. The fixed input used here establishes neither physical sensor freshness nor the ability to detect a real temperature excursion.

The memory and prediction units depend on the acquisition history. A 100-sample window spans roughly ten seconds only while samples arrive near 10 Hz; gaps or rejected inputs extend the elapsed time represented by the retained history. The implementation stores timestamps and uses their span in analysis, which is preferable to assuming uniform arrivals, but timestamps cannot reconstruct a missing excursion. A delay in one unit can therefore affect what the next unit knows about the room, even when every numerical operation remains correct.

B. Prioritize the service rather than a label

My notes favor giving sampling and reporting precedence over background computation. The competition results support protecting their response margin. However, the current program has no separate sensing, storage, or prediction threads to prioritize independently. Raising its process preference raises the whole measured path, including operations that can block. It would not remove the acquisition and saving stalls observed in the low-RAM run.

For a future version that runs prediction continuously, I would first establish how its resource demand interferes with acquisition, retained history, and reporting. Any proposed separation of these activities would need explicit limits on queues, shared resources, and report age. A high-priority sampler could still wait for a shared lock or for data managed by lower-priority work. The relevant objective is timely delivery of the sensing service through its dependencies, not merely assigning a favorable numeric priority.

C. Treat resource reduction and distributed operation as candidates

The one-core and powersave results make lower-resource operation a reasonable candidate for the present workload. They address part of the timing question left open by my power paper [3]. They do not certify that paper's combined configuration: the

timing campaign did not combine every power setting, edited assembly, and disabled peripheral. Individual energy savings and isolated timing margins should not be added or assumed to survive in combination.

Likewise, the network trials are encouraging for a distributed Critter installation because moderate receive traffic did not disrupt the tested acquisition schedule. They do not measure end-to-end delivery of a temperature warning, outgoing report bursts, communication loss, or several devices contending for a shared service. A room may be sampled on time while its operator sees old information. The broader application of this result is to distinguish local acquisition predictability from system-level information freshness.

D. Bound memory use in the recurring path

My concern about accumulating data connects directly to both safety and timing. Unbounded retention or pending reports can turn a small monitoring task into a system-memory problem. The current history ring is bounded, but code inspection shows that outlier detection still allocates and frees a temporary temperature array during recurring filtering. Initializing the history and timing trace before collection does not remove all later allocation. No operation failure was recorded, and the measured large stalls occurred in I/O, so this allocation is not established as their cause.

I would favor a fixed, initialized workspace for recurring analysis and explicit storage bounds for any future communication backlog. Holzmann's rule against dynamic allocation after initialization is relevant because it improves the ability to bound and analyze critical memory use [9]. It does not by itself prevent paging of other resources, bound file-system service, or guarantee deadlines under general-purpose Linux. The lesson is to make the entire required path and its resource limits analyzable, rather than treating one coding rule as a complete timing argument.

V. Broader Embedded Systems Implications

The pacemaker example in my notes makes the consequence of a deadline part of the design decision. A proposed device might give sensing and electrical-stimulation work higher precedence than Bluetooth telemetry. The Critter results illustrate why competing communication or computation should not receive the same treatment as the critical service without analysis. They do not show that a pacemaker can safely combine those activities on one core, or that an observed 100 ms requirement is appropriate for cardiac operation.

A single-core design could be feasible for some specified workload, but feasibility would depend on target-specific worst-case execution, interrupt interference, blocking, shared memory, and communication behavior. A higher-priority function can still be delayed by its dependencies. Increasing the core count also does not automatically remove contention for memory or peripherals. This follows the same reasoning as Critter's sequential pipeline, but the acceptable evidence is different when a missed sensing or stimulation deadline can violate a safety requirement.

Extensive testing under expected and adverse conditions is necessary evidence, yet observing no misses is not a proof that every possible deadline will be met. For hard real-time behavior, I would combine target measurements with bounded execution and interference arguments, explicit failure behavior, and validation of the device in its environment. Jiang, Pajic, and Mangharam's closed-loop pacemaker work illustrates why temporal requirements must be evaluated with the heart's state and operating conditions, rather than with software inputs generated independently of that environment [10].

The low-RAM event reinforces my concern about uncontrolled data growth in such systems. Fixed capacities and controlled allocation can make the required memory envelope easier to reason about, but do not establish a safe device on their own. Critical buffers, stack use, communication queues, and failure paths must all remain within analyzed limits. The transferable lesson is not that memory is always the largest risk or that every service needs maximum priority. It is that the consequence of a missed deadline determines which delays must be bounded and how strong the supporting evidence must be.

VI. Limitations and Further Validation

Three one-minute repetitions per condition provide limited exposure to rare events. Core and RAM runs occur in successive boots, so boot state, background work, and time can confound comparisons. One-second telemetry can document resource activity without resolving the immediate cause of a subsecond stall. There is no demonstrated relationship between all delay events and paging, thermal state, or a particular kernel operation. The observed maxima are measurements, not worst-case bounds.

The fixed CSV input and small history simplify I/O and memory demand; the brief predictor runs after acquisition. The results should therefore not be generalized to physical sensor transactions, a large historical dataset, or continuous computationally intensive forecasting. Generated receive traffic is only one network behavior, and the primary condition levels have unequal stress severity. The study ranks the effects it actually tested, not the inherent importance of CPU scheduling, memory, and networking in all embedded systems.

My next priority would be repeating the low-RAM condition with matched background state and more detailed I/O/fault timing, followed by a combined one-core, powersave, and network trial. Representative temperature ramps and interruptions would then connect timing gaps to retained history and report freshness. Those studies would test whether the current candidates remain useful to the real Critter product; none is needed to replace the central observation that dispatch interference and I/O stalls require different explanations.

VII. Conclusion

Within the tested Critter workload, matrix competition and scheduling preference produced the largest repeatable change in response time, while one 1.5 GiB run produced the most severe observed deadline failure. Nice +19 increased mean median response to 3.128 ms without missing a deadline. The low-RAM

event produced six missed slots, 245.159 ms maximum lateness, a four-slot miss sequence, and a 444.746 ms acquisition gap despite a sub-millisecond p99.

For Critter, the practical result is a set of resource-reduction candidates and a stronger definition of what reliable monitoring must preserve: timely acquisition, useful history, and information that reaches the operator while it is still relevant. Priority can protect dispatch margin but cannot guarantee a path that blocks inside I/O. The pacemaker comparison extends that reasoning to hard real-time work, where favorable averages and extensive tests must be supported by bounded timing and environmental validation. Predictability is determined by the required service and its dependencies, not by a single fast average or configuration setting.

Acknowledgment

This study uses ECE 2160 Group C's Critter prototype. Generative AI assisted with instrumentation, data analysis, and preparation of the manuscript using my experimental findings, annotations, and interpretations.

References

[1] University of Pittsburgh, ECE 2160, “Reliability homework and Critter product description,” unpublished course assignment, Fall 2026. Assignment text supplied with this study.

[2] M. Ketas and ECE 2160 Group C, “Critter timing assessment,” unpublished course-project materials, Oct. 2026. Sources: critter-timing-results.tar.gz; annotated critter_timing_review.xlsx; TimingAssessment collector, analyzer, and C harness; and Pilot I/O, memory, and computation source. Primary analysis: 48 completed runs in seven sessions; workbook annotations dated Oct. 23, 2026.

[3] M. Ketas, “Evaluating Power Reduction for the Critter Embedded System,” ECE 2160 homework paper, University of Pittsburgh, Sep. 2026. Author-supplied manuscript.

[4] Linux man-pages project, “clock_nanosleep(2),” Linux manual pages. Accessed: Oct. 23, 2026. [Online]. Available: https://man7.org/linux/man-pages/man2/clock_nanosleep.2.html

[5] Linux man-pages project, “time(1),” sections “GNU version” and resource formats. Accessed: Oct. 23, 2026. [Online]. Available: https://man7.org/linux/man-pages/man1/time.1.html

[6] The Linux Kernel, “CPU Performance Scaling,” policy attributes and generic governors. Accessed: Oct. 23, 2026. [Online]. Available: https://www.kernel.org/doc/html/latest/admin-guide/pm/cpufreq.html

[7] ESnet, “Invoking iperf3,” iperf3 documentation. Accessed: Oct. 23, 2026. [Online]. Available: https://software.es.net/iperf/invoking.html

[8] Linux man-pages project, “sched(7),” sections on SCHED_OTHER, nice values, and autogroups. Accessed: Oct. 23, 2026. [Online]. Available: https://man7.org/linux/man-pages/man7/sched.7.html

[9] G. J. Holzmann, “The Power of Ten—Rules for Developing Safety Critical Code,” NASA/JPL Laboratory for Reliable Software, 2006, rule 3. [Online]. Available: https://spinroot.com/gerard/pdf/P10.pdf

[10] Z. Jiang, M. Pajic, and R. Mangharam, “Model-based closed-loop testing of implantable pacemakers,” in Proc. IEEE/ACM Int. Conf. Cyber-Physical Systems, 2011, pp. 131–140, doi: 10.1109/ICCPS.2011.28.